Google Maps Integration

Roswalt My Home — Employee Live-Tracking Map

Reference for integrating the same Google Maps live-tracking feature into another mobile application. Source: Roswalt My Home Flutter app (employee module).

1. Dependencies (pubspec.yaml)

```
google_maps_flutter: ^2.10.0
geolocator: ^13.0.1
```

2. API Key

Android — AndroidManifest.xml, inside <application> (lines 19-21):

```
<meta-data
    android:name="com.google.android.geo.API_KEY"
    android:value="AIzaSyCYLkqkzpEQ5ocGQyfI0k6ShZREZpPNlwE" />
```

iOS — not configured in the source app. No GMSServices.provideAPIKey(...) call exists in AppDelegate.swift, and no GMSApiKey entry exists in Info.plist. If the new app targets iOS, add:

```
// AppDelegate.swift, before super.application(...)
import GoogleMaps
GMSServices.provideAPIKey("YOUR_IOS_KEY")
```

Important: This Android key should carry application restrictions (package name + SHA-1 fingerprint) in Google Cloud Console. It will very likely NOT work unmodified for a different app with a different package name / SHA-1. Recommended: generate a new, separately-restricted key for the new app rather than reusing this one as-is. Use a distinct, HTTP-referrer or bundle-ID restricted key per platform/app.

3. Permissions

Android — AndroidManifest.xml (lines 6-7):

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
```

iOS — not present in Info.plist. Add NSLocationWhenInUseUsageDescription (and NSLocationAlwaysAndWhenInUseUsageDescription if background tracking is needed) with a user-facing purpose string before shipping on iOS.

4. Firestore Data Model (location trail)

GPS points are streamed live from Firestore, not held only in memory. Two parallel trail collections exist, same shape:

Purpose	Firestore path	Ordered by
Daily attendance movement	employees/{uid}/attendance/{yyyy-mm-dd}/locationTrail	recordedAt

Field Duty session	employees/{uid}/fieldDuty/status/trail	recordedAt

Each point document holds numeric `latitude` / `longitude` fields, mapped client-side to a `LatLng`.

5. Implementation Pattern (lib/screens/employee/employee_live_map_screen.dart)

Three widgets follow the same pattern — live Firestore stream → trail of LatLng → GoogleMap with marker(s) + polyline:

a) EmployeeMovementMapCard — today's check-in-to-now trail, embedded above the Attendance Calendar.

b) EmployeeFieldDutyMapCard — compact dashboard card, shown while a Field Duty session is active; tapping it opens the full-screen version.

c) EmployeeOwnFieldDutyMapScreen — full-screen, interactive version of the same Field Duty trail.

Core Firestore listener pattern (same shape in all three):

```
final uid = FirebaseAuth.instance.currentUser?.uid;
_trailSub = FirebaseFirestore.instance
    .collection('employees').doc(uid)
    .collection('attendance').doc(dateKey)   // or fieldDuty/status
    .collection('locationTrail')             // or 'trail'
    .orderBy('recordedAt')
    .snapshots()
    .listen((snap) {
  final points = snap.docs.map((d) => LatLng(
      (d['latitude'] as num).toDouble(),
      (d['longitude'] as num).toDouble())).toList();
  setState(() => _trail = points);
  if (points.isNotEmpty) {
    _mapController?.animateCamera(CameraUpdate.newLatLng(points.last));
  }
});
```

GoogleMap widget configuration:

```
GoogleMap(
  initialCameraPosition: CameraPosition(target: current, zoom: 16),
  onMapCreated: (c) => _mapController = c,
  myLocationEnabled: true,
  zoomControlsEnabled: false,
  markers: {
    Marker(markerId: MarkerId('start'), position: trail.first,
        icon: BitmapDescriptor.defaultMarkerWithHue(BitmapDescriptor.hueGreen),
        infoWindow: InfoWindow(title: 'Check-in location')),
    Marker(markerId: MarkerId('current'), position: current,
        icon: BitmapDescriptor.defaultMarkerWithHue(BitmapDescriptor.hueAzure),
        infoWindow: InfoWindow(title: 'Current position')),
  },
  polylines: {
    if (trail.length > 1)
      Polyline(polylineId: PolylineId('trail'), points: trail,
          color: Color(0xFF543813), width: 4),
  },
)
```

The compact dashboard card variant disables all gestures (scrollGesturesEnabled/zoomGesturesEnabled/rotateGesturesEnabled/tiltGesturesEnabled: false) and

wraps the map in an IgnorePointer, since it's a tap target that opens the full-screen map rather than an interactive map itself.

6. GPS Capture (write side) — not included here

This document covers the read/display side only (the map screens). The code that actually captures GPS pings and writes them into locationTrail / fieldDuty/status/trail lives in EmployeeAttendanceProvider (method _startLiveTracking), using the geolocator package, and runs only in the foreground while an employee is checked in. Request that separately if you need the capture/write logic as well as the display logic.

7. Integration Checklist for the New App

- Add google_maps_flutter and geolocator to pubspec.yaml
- Generate a NEW Google Maps API key restricted to the new app's package name + SHA-1 (Android) and/or bundle ID (iOS) — do not reuse the key above as-is
- Add the Android key via in AndroidManifest.xml
- For iOS, call GMSServices.provideAPIKey(...) in AppDelegate.swift with a separate iOS key
- Add ACCESS_FINE_LOCATION / ACCESS_COARSE_LOCATION permissions on Android
- Add NSLocationWhenInUseUsageDescription (and Always, if needed) to iOS Info.plist
- Define your Firestore trail collection(s) with latitude/longitude/recordedAt fields, ordered by recordedAt
- Implement the GoogleMap + Firestore snapshot listener pattern shown in section 5
- Apply Firestore security rules scoping each trail subtree to its owning user's uid, mirroring the existing employees/{uid}/** rule